

Live-Testbed Realization of QoE-Aware Deep Reinforcement Learning for O-RAN Network Slice Admission Control

Manoj Kumar Palanisamy **TODO(MANOJ):** confirm exact name/order and any co-authors

TODO(MANOJ): affiliation, university, email

Abstract—

I. INTRODUCTION

TODO(MANOJ): Abstract. Suggested content sketch (2-3 bullets):

- First live-testbed (not simulation) realization of DRL-based O-RAN slice admission control from papers #1/#2, on a real OAI gNB with the real E2 control-plane wire protocol – prior work was ns-3/ns-O-RAN simulation only.
- Headline quantitative result once Section V numbers are final (mean $\pm$ std across 3 seeds): SLA compliance / backlog comparison, static baseline vs. DQN/A2C under SLA and QoE-aware rewards.
- Testbed engineering contribution: empirical ratio-to-PRB calibration methodology, admission-as-ceiling-nudging realization, and a documented rig-fragility mitigation – reusable findings for anyone building a live O-RAN RL testbed.

TODO(MANOJ): Introduction. Suggested content sketch:

- Motivation: O-RAN network slicing needs adaptive admission control; DRL is a natural fit but nearly all prior evaluation (including our own papers #1/#2) is simulation-only (ns-3/ns-O-RAN).
- Gap: no live-testbed validation of whether simulation-trained admission policies transfer to a real E2 control loop, a real MAC scheduler, and real per-slice traffic.
- Contribution list (map directly to Sections III-VI): (1) a working live O-RAN DRL admission-control testbed on real OAI hardware; (2) **a control-surface mismatch finding, promoted here from a Section III implementation note to a first-class contribution:** real OAI's E2 agent exposes no per-flow accept/reject hook at all – the only control primitive is a per-slice PRB ratio ceiling/floor (`slicing_control_m`), so any DRL admission policy from papers #1/#2 must be realized as *ceiling nudging*, not literal accept/reject, on any OAI-based live rig; the ratio-to-PRB mapping this ceiling controls is also *not* 1:1 (empirically non-linear, Section III-C) – both are structural findings for anyone building a live O-RAN RL testbed on this platform, not just an implementation detail of this paper; (3) a live comparison of a static O-RAN-native baseline vs. DQN/A2C under SLA (papers #1/#2 eq.) and QoE-aware (eq. 9) rewards; (4) an honest characterization of what does and does not hold up on real hardware (Section VII).
- Scope statement: single gNB, 3 UEs (one per slice), rfsim. Note the three slices already contend for one shared 106-PRB pool at this single gNB – a genuine (if small-scale) multi-agent coordination problem, not the absence of one (see Related Work correction below). GNN state representation, an explicit multi-agent (CTDE) controller over that shared pool, multi-gNB coordination, and federated learning across gNBs (paper #3's remaining scope) are explicitly future work for the journal-length follow-up, not attempted here.

*Index Terms—*O-RAN, network slicing, deep reinforcement learning, admission control, quality of experience, live testbed

II. RELATED WORK

TODO(MANOJ): Related work. Suggested content sketch:

- Position against papers #1 (MECON) and #2 (ICRAIE): same admission-control MDP formulation and reward (eq. 2), but simulation-only there vs. live here.
- Position against paper #3 (the IEEE Access survey): this work is a grounded first step toward the survey’s live-testbed vision, deliberately scoped down (no GNN/MARL/FL yet). **Correction from an earlier draft:** do NOT claim a single gNB has no multi-agent coordination problem – it does: three slices (eMBB/URLLC/mMTC) contend for one shared 106-PRB pool at this one gNB, and each of the three per-slice admission decisions in Section III-B affects the ceiling headroom available to the other two. That shared-resource contention is exactly the kind of coordination problem a GAT-CTDE controller (one agent per slice, centralized critic over the joint state) targets – this paper’s single-agent-per-slice-independently arms are a deliberately simplified special case of that problem, not evidence the problem is absent. Frame the journal-length follow-up’s GAT-CTDE contribution as resolving this coordination problem explicitly, rather than as solving a problem this paper claims doesn’t exist.
- Brief coverage of O-RAN near-RT RIC / xApp literature, and any other live O-RAN RL testbeds you’re aware of (if none exist with a real E2 loop, that itself is worth stating explicitly).

III. TESTBED AND IMPLEMENTATION

A. Architecture

The testbed comprises a real OpenAirInterface (OAI) gNB running in radio-frequency simulation (rfsim) mode, an Open5GS 5G core provisioned with three slice-specific SMF/UPF pairs, and three OAI `nr-uesoftmodem` UE instances, one per slice (eMBB, URLLC, mMTC), each attached to its own provisioned S-NSSAI. Fig. 1 shows the control-loop architecture. The gNB is built from the ORANSlice fork of OAI (2024.w28 base) [1], which integrates a RIC-free, UDP-based E2 agent directly into the gNB binary – distinct from vanilla upstream OAI, whose only E2 agent (FlexRIC) requires a full E2AP/SCTP near-RT RIC deployment [2], [3].

B. The E2 Control Loop and the Admission-as-Ceiling-Nudging Realization

The gNB’s E2 agent exposes a request/response UDP protocol (ports 6655/6600): an `INDICATION_REQUEST` returns one `INDICATION_RESPONSE` carrying per-UE KPM samples (`avg_prbs_dl`, `dl_mac_buffer_occupation`, `dl_bler`, ...); a `CONTROL` message is fire-and-forget. Critically, *the only control primitive this E2 agent exposes is* `slicing_control_m{sst, sd, min_ratio,`

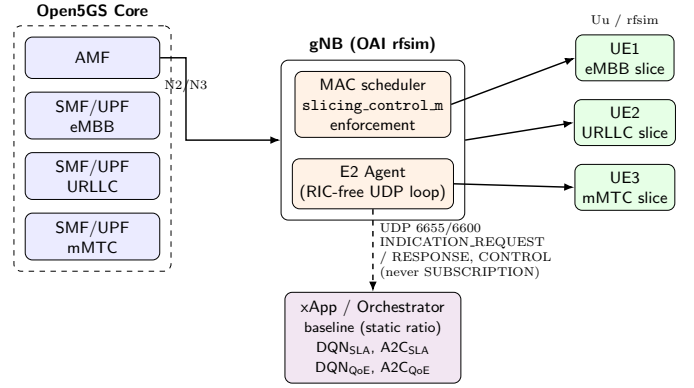


Fig. 1: Testbed control-loop architecture: Open5GS core, OAI gNB (MAC scheduler + E2 Agent), 3 rfsim UEs (one per slice), and the xApp/orchestrator realizing all 5 evaluated arms.

`max_ratio` – a per-slice PRB ratio ceiling/floor enforced by the MAC scheduler [4]. There is no raw per-flow accept/reject hook. Consequently, the admission-control MDP of papers #1/#2 – whose action space is a binary accept/reject decision on an incoming request – is realized here as *ceiling nudging*: an accept decision raises a slice’s `max_ratio` toward its calibrated cap, a reject lowers it toward its floor. Capping a slice starves it (demand becomes backlog); it does not remove the demand. This is a structural constraint of any OAI-based live testbed, not an implementation choice, and every result in Section V should be read through this lens.

C. Empirical Ratio-to-PRB Calibration

A first-principles assumption – that the `max_ratio` configuration parameter (0–100 scale) maps linearly onto real PRBs at this cell’s 106-PRB/ $\mu = 1$ configuration – was tested directly against live measurement and found false: pinning eMBB’s `max_ratio` to 4 empirically permitted 15–22 real served PRBs, not 4. Ratio caps for all three slices were therefore calibrated by direct measurement rather than by formula: real per-UE offered demand was polled with the ceiling held wide open (`min_ratio=0`, `max_ratio=100`), and each slice’s cap was then set below its own measured organic demand (Table II), reproducing the intended contention regime empirically instead of assuming it. A dedicated contention-gate test (Table I) confirmed the calibration is load-bearing: pinning eMBB’s ceiling to its floor for ~ 45 s raised `dl_mac_buffer_occupation` (bytes of queued RLC TX buffer, summed across the slice’s UEs – not a PDU count) by more than four orders of magnitude ($411 \rightarrow 6.6 \times 10^6$ bytes), and restoring a ceiling closer to (but still below) organic demand produced real, measured drainage ($6.6 \times 10^6 \rightarrow 2.5 \times 10^6$ bytes) rather than permanent saturation – i.e., ceiling changes have real, reversible, non-trivial consequences on this rig, which is the precondition for any of the policy comparisons in Section V to be meaningful.

TABLE I: Contention gate: eMBB backlog (mean `dl_mac_buffer_occupation`, bytes) under ceiling manipulation

Phase	Ceiling (min,max)	Mean backlog (bytes)
Baseline (wide open)	(0, 100)	411.3
Pinned (starved)	(1, 1)	6,601,079.0
Recovery (restored)	(1, 20)	2,504,530.4

D. Baseline Realization

The comparison’s static baseline is intended to model the “heuristic, built-in, non-xApp” O-RAN default: a fixed per-slice ratio, no learning, no per-step control. The framework’s own `rrmPolicy.json` periodic-reload path, which would provide this natively, was confirmed non-functional on this OAI checkout (the reload code path is compiled out unless a source patch is applied). The baseline is instead realized by pinning each slice’s admission-gate ceiling-step size to zero, so no accept/reject decision can move the ceiling away from its calibrated nominal ratio for the entire episode – verified live: the commanded ceiling is bit-for-bit identical on every step of every baseline episode (Fig. 2).

E. Learned Arms

Two algorithms (DQN, A2C) are each trained offline under two reward formulations: the SLA-only reward of papers #1/#2 (Eq. 2) and a QoE-aware reward $r = \alpha \cdot \text{MOS} - \beta \cdot \text{cost} - \gamma \cdot \text{SLA_viol}$ (Eq. 9, $\alpha=1.0, \beta=0.2, \gamma=0.5$), where MOS is inferred per slice via an IQX closed-form prior [5] refined by a small per-slice LSTM, calibrated against ITU-T P.1203 [6] objective labels for eMBB and ACR-style task-success scoring for URLLC/mMTC. All four learned arms are trained offline against a closed-loop synthetic KPM source (demand response to admission decisions, not open-loop) for 300 episodes across 3 seeds, then evaluated live with frozen weights – live episodes never update policy parameters.

IV. EXPERIMENTAL SETUP

Table II summarizes the fixed testbed and campaign parameters. All five arms (baseline, DQN_{SLA} , A2C_{SLA} , DQN_{QoE} , A2C_{QoE}) are evaluated against the identical per-slice traffic profiles, episode length, and evaluation-seed set, with arm order interleaved across seeds (not run back-to-back per arm) to decorrelate any single arm’s results from time-dependent rig drift. Ceilings are reset to a wide-open state and backlog is drained (verified by re-probing real demand) between every arm.

TODO(MANOJ): Note once Section V is finalized: if the actual completed grid differs from the 3 seeds $\times$ 5 episodes $\times$ 5 arms planned here (e.g., an arm fell short after retries per the campaign’s own stop conditions), Table II and this section must be updated to state what was actually run, not what was planned.

V. RESULTS

A. Ceiling mechanism and SLA compliance

Fig. 2 shows the mechanism directly: baseline’s commanded ceiling is bit-for-bit flat at its static nominal ratio for the entire episode, while DQN (SLA reward) rides eMBB’s ceiling from floor to the calibrated cap (12) within ~ 13 steps and jumps URLLC/mMTC to their caps (4, 3) immediately, holding both for the remainder. The direct consequence is Fig. 3: all four learned arms ($\text{DQN/A2C} \times \text{SLA/QoE}$ reward) hold $100.0 \pm 0.0\%$ SLA compliance (bootstrap 95% CI $[100.0, 100.0]$) on all three slices, in every one of 45 learned-arm episodes (15/15 fully-compliant episodes per slice), against baseline’s $73.6 \pm 18.6\%$ (CI $[53.4, 93.4]$), 11/15 fully-compliant episodes per slice – with zero losses against the baseline in any single slice $\times$ seed comparison of per-seed means. We do not report Cohen’s d here: every learned arm’s per-seed compliance is exactly 100.0% on all 3 seeds (zero variance), so a pooled-variance effect size would be driven entirely by baseline’s own spread and would not mean what a reader expects it to. A Wilcoxon signed-rank test on the underlying per-episode series (paired by seed and episode index, $n = 15$ pairs/slice) does *not* clear $p < 0.05$ for any slice ($p \approx 0.059\text{--}0.066$) – because baseline is already fully compliant on 11/15 of the same episodes, only the ~ 4 discordant episodes per slice carry any signal for a paired rank test, and $n = 15$ is too small to reach significance on so few informative pairs. We report this honestly rather than lead with an underpowered p -value: the categorical, practically decisive finding is the worst-episode comparison below, not the signed-rank test. Table III reports the full per-arm, per-slice breakdown, including worst-episode compliance and P5 margin alongside the means: baseline’s *worst* single episode is 0.0% compliant on every slice, against 100.0% for every learned arm’s worst episode – the mean $\pm$ std view alone understates how categorical this gap is, and unlike the signed-rank test, this worst-case comparison does not depend on how many episodes happen to tie.

Baseline’s compliance is real but *seed-dependent*, not uniformly poor: 60.3% (seed 950), 100.0% (seed 951), 60.7% (seed 952) – an initial single-episode pilot run had sampled only the lower mode and suggested near-zero compliance, which did not hold up once seeded episodes were run at scale. The corrected, honest reading is that the static ratio is sometimes adequate and sometimes not depending on traffic/timing, whereas every learned arm is consistently at ceiling regardless of seed – the practical value demonstrated here is *reliability under variable conditions*, not rescuing an otherwise-always-failing baseline.

B. Backlog and the contention gate at campaign scale

Fig. 4 shows the per-slice SLA margin (a continuous, Lmax-normalized backlog-severity proxy; raw values reach $\approx -10^6$ under baseline’s worst contention). The top row plots one representative episode’s margin trajectory (seed 950, episode 1; baseline vs. DQN/SLA) on a symlog axis, the only way to render a $+1.0$ -margin trajectory and a -10^6 -margin trajectory on shared axes – baseline collapses to its

TABLE II: Testbed and campaign parameters

Parameter	Value
<i>RAN configuration</i>	
gNB	OAI rfsim, ORANSlice fork (OAI 2024.w28 base)
Bandwidth / numerology	106 PRB / $\mu = 1$ (30 kHz SCS), band 78
E2 control primitive	slicing_control_m{sst,sd,min_ratio,max_ratio}
E2 transport	UDP 6655/6600, request/response (no SUBSCRIPTION)
<i>Slices (S-NSSAI)</i>	
eMBB	SST=1, SD=0xFFFFF
URLLC	SST=1, SD=0x000001
mMTC	SST=1, SD=0x000002
<i>Traffic profiles (per slice, fixed, never varied across arms)</i>	
eMBB	sustained UDP, 4 Mbps, 1200 B packets
URLLC	sustained UDP, 300 kbps, 100 B packets
mMTC	bursty UDP, 50 kbps, 80 B packets, 2 s on / 6 s off
<i>Calibrated PRB ratio ceilings (empirically derived, §IV)</i>	
eMBB max_ratio_cap	12 (organic demand $\approx$ 15 PRB mean)
URLLC max_ratio_cap	4 (organic demand $\approx$ 5 PRB floor)
mMTC max_ratio_cap	3 (organic demand $\approx$ 5 PRB floor)
<i>QoE reward (eq. 9) weights</i>	
α (MOS) / β (cost) / γ (SLA viol.)	1.0 / 0.2 / 0.5
<i>Evaluation campaign</i>	
Arms	baseline, DQN _{SLA} , A2C _{SLA} , DQN _{QoE} , A2C _{QoE}
Episode length	60 steps $\times$ 5 s = 5 min
Seeds (paired across arms)	950, 951, 952
Episodes / arm / seed	5
Total live episodes	75
Offline training (per learned arm)	300 episodes $\times$ 3 seeds (256, 257, 258)

floor within the first ~ 5 steps and never recovers, while the learned arm stays flat near +1.0 for the entire episode. The bottom row keeps the original pooled CDF across all 3 seeds $\times$ 5 episodes (display-clipped at -1.5): baseline sits at the clip floor for nearly the entire pooled distribution, while all four learned arms sit mostly in the comfortable region (≈ 0.7 – 1.0). Together these connect Phase 1’s isolated contention-gate demonstration (Table I) to the campaign’s actual multi-seed, multi-arm results, both as a single concrete event and as an aggregate.

C. Inferred MOS: a dissociated, largely policy-independent signal

Fig. 5 shows inferred per-slice MOS. eMBB and URLLC sit low (≈ 1.2 – $1.7/5$) and are overwhelmingly policy-independent – eMBB MOS is identical to three decimal places across all four learned arms (1.223 ± 0.000) – while mMTC sits high (≈ 4.6 – $4.8/5$), equally policy-independent. This is not a contradiction of the compliance result: SLA compliance reflects the learned policy successfully managing the backlog/loss budget, while MOS reflects a separately-calibrated QoE inference that this traffic profile and QoE mapper do not map to a high score for these two slices regardless of control policy – meeting the SLA budget is not the same as achieving high inferred QoE, which is exactly the gap a QoE-aware reward is meant to surface. Baseline’s URLLC MOS variance (std 0.331) is by far the largest of any arm/slice (next-highest: 0.120), consistent with – and mechanistically tied to – its seed-dependent compliance above.

D. A systematic anomaly: A2C+QoE-reward mass rejection

Fig. 6 shows per-episode URLLC *agent-issued rejections* pooled across seeds – a reject decision the admission-gate policy makes internally (Section III-B); there is no corresponding dropped-PDU event on this rig, since the E2 agent exposes no per-flow accept/reject hook. Baseline and three of four learned arms (DQN/SLA, A2C/SLA, DQN/QoE) show *exactly zero* such rejections in every one of their 45 episodes combined. A2C/QoE is a clear exception: every single one of its 15 episodes (all 3 seeds) rejects 28–49 requests on *all three slices simultaneously*, while its algorithm-matched SLA control (A2C/SLA) and its reward-matched DQN control (DQN/QoE) both show zero rejections throughout. Applying a pre-registered decision rule (systematic if > 10 rejections/episode on ≥ 2 slices recurs in $\geq 1/3$ of episodes across ≥ 2 seeds) to this data yields an unambiguous *systematic* verdict – this behavior clears the threshold by a wide margin (100% of episodes, all 3 slices, all 3 seeds). The effect is specific to the A2C+QoE-reward combination, not general to either factor alone. Despite the heavy rejection rate, A2C/QoE still achieves full SLA compliance and the *highest* mean episodic reward of the two QoE-reward arms (0.388 vs. DQN/QoE’s 0.318) – the eq. 9 reward does not clearly penalize this behavior, a reward-design tension we return to in Section VII rather than a defect in the experimental campaign.

TABLE III: Per-arm, per-slice results (mean $\pm$ std across 3 seeds); Worst ep. and P5 margin are pooled across all episodes/steps, not means-of-means. “Agent-issued rej.” is an internal admission-gate reject-decision count, not a measured dropped PDU (no per-flow accept/reject hook exists on this rig; see Sec. III-B).

Arm	Slice	SLA compliance (%)	Worst ep. (%)	P5 margin	Agent-issued rej./ep.	Backlog margin	Inferred MOS
baseline (static)	embb	73.7 $\pm$ 18.6	0.0	-1e+06	0.0 $\pm$ 0.0	-257627.35 $\pm$ 182170.84	1.26 $\pm$ 0.02
	urllc	73.4 $\pm$ 18.8	0.0	-1.02e+06	0.0 $\pm$ 0.0	-223096.26 $\pm$ 157753.41	1.24 $\pm$ 0.33
	mmtc	73.8 $\pm$ 18.5	0.0	-810	0.0 $\pm$ 0.0	-147.81 $\pm$ 129.86	4.65 $\pm$ 0.09
DQN (SLA reward)	embb	100.0 $\pm$ 0.0	100.0	1	0.0 $\pm$ 0.0	1.00 $\pm$ 0.00	1.22 $\pm$ 0.00
	urllc	100.0 $\pm$ 0.0	100.0	0.7	0.0 $\pm$ 0.0	0.73 $\pm$ 0.00	1.49 $\pm$ 0.05
	mmtc	100.0 $\pm$ 0.0	100.0	0.7	0.0 $\pm$ 0.0	0.74 $\pm$ 0.01	4.78 $\pm$ 0.00
A2C (SLA reward)	embb	100.0 $\pm$ 0.0	100.0	1	0.0 $\pm$ 0.0	1.00 $\pm$ 0.00	1.22 $\pm$ 0.00
	urllc	100.0 $\pm$ 0.0	100.0	0.7	0.0 $\pm$ 0.0	0.75 $\pm$ 0.01	1.66 $\pm$ 0.12
	mmtc	100.0 $\pm$ 0.0	100.0	0.7	0.0 $\pm$ 0.0	0.73 $\pm$ 0.00	4.77 $\pm$ 0.00
DQN (QoE reward)	embb	100.0 $\pm$ 0.0	100.0	1	0.0 $\pm$ 0.0	1.00 $\pm$ 0.00	1.22 $\pm$ 0.00
	urllc	100.0 $\pm$ 0.0	100.0	0.7	0.0 $\pm$ 0.0	0.74 $\pm$ 0.01	1.55 $\pm$ 0.06
	mmtc	100.0 $\pm$ 0.0	100.0	0.7	0.0 $\pm$ 0.0	0.73 $\pm$ 0.00	4.77 $\pm$ 0.00
A2C (QoE reward)	embb	100.0 $\pm$ 0.0	100.0	0.7	42.5 $\pm$ 1.0	0.73 $\pm$ 0.00	1.22 $\pm$ 0.00
	urllc	100.0 $\pm$ 0.0	100.0	0.7	41.2 $\pm$ 1.9	0.75 $\pm$ 0.01	1.66 $\pm$ 0.06
	mmtc	100.0 $\pm$ 0.0	100.0	0.7	37.5 $\pm$ 2.9	0.72 $\pm$ 0.01	4.77 $\pm$ 0.01

Commanded PRB ceiling: baseline vs. best learned arm

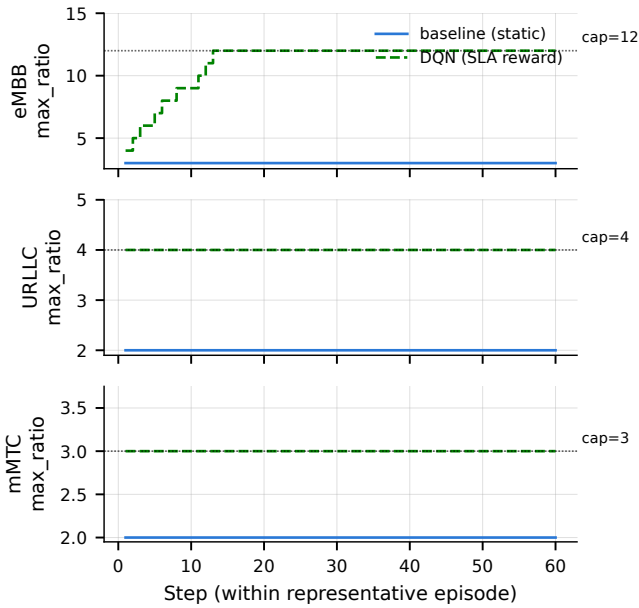


Fig. 2: Commanded PRB ceiling per slice over one representative episode (seed 950): baseline (flat) vs. DQN/SLA (rides to the calibrated cap, annotated per slice).

E. Sim-to-real transfer parity: informative for one slice, uninformative for two

All four learned arms are trained offline, then deployed live with frozen weights (Section III-E); Fig. 7 compares each arm’s offline-predicted SLA compliance (mean over each training seed’s final 5 rollup episodes, pooled across the 3 training seeds – the same 5-episode window size used live) against its live-measured compliance from Table III. The

Per-episode SLA compliance by arm ($n=15$ episodes/arm, 3 seeds)

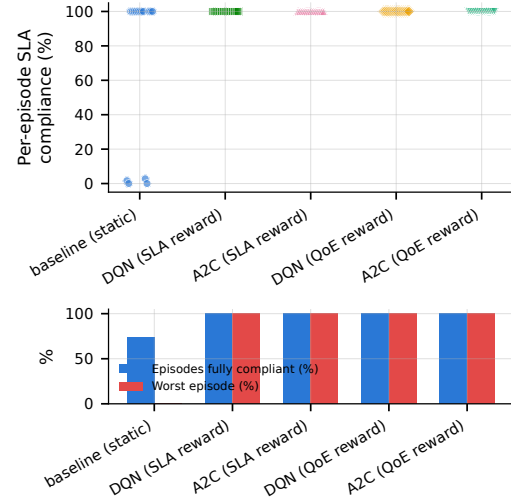


Fig. 3: Per-episode SLA compliance (%) by arm, $n = 15$ episodes/arm (3 seeds $\times$ 5 episodes). Top: one point per episode – every learned arm sits at exactly 100.0% in all 15 of its episodes, while baseline shows a genuine bimodal split (a majority near 100%, a real cluster at 0%) rather than the uniform mean the original mean $\pm$ std bar figure implied. Bottom: fraction of episodes fully SLA-compliant and worst single-episode compliance, per arm – baseline’s worst episode is 0.0% compliant against 100.0% for every learned arm.

result is not a clean diagonal. eMBB transfers exactly: 100.0% offline $\rightarrow$ 100.0% live, for every one of the four arms. URLLC and mMTC do not: every arm’s *offline*-predicted compliance for these two slices is 0.0% – not near zero, exactly zero, on every one of the 300 training episodes for every arm, with the per-step SLA margin sitting at ≈ -19 throughout (deeply past budget by the offline environment’s own accounting) –

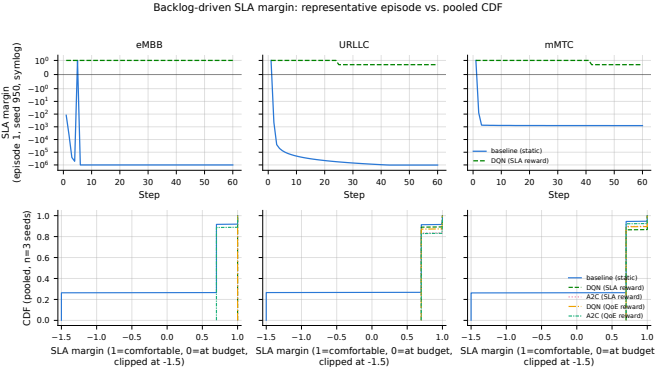


Fig. 4: Top: per-slice SLA margin over one representative episode (seed 950, episode 1), baseline vs. DQN/SLA, symlog axis. Bottom: CDF of per-slice SLA margin by arm, pooled across 3 seeds $\times$ 5 episodes.

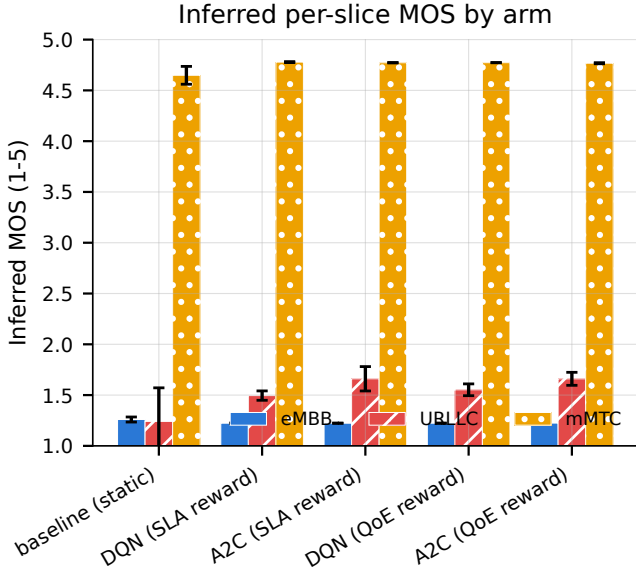


Fig. 5: Inferred per-slice MOS, mean $\pm$ std, $n = 3$ seeds.

while the same frozen policies measure 100.0% live on both slices. This is not an accidental scale mismatch: the offline training config (`sac1b_offline_campaign.yaml`) deliberately pins URLLC/mMTC’s `max_ratio_cap` at their own `nominal_ratio` (zero elastic ceiling headroom), by design keeping them permanently below `ClosedLoopKpmSource`’s synthetic demand – an oversubscription-by-construction training regime, documented in the config itself, that never intended offline compliance to be a live predictor for these two slices in the first place. The consequence stands regardless of intent: the SLA-only reward (Eq. 2) had no compliance-based learning signal to differentiate policies on URLLC/mMTC during training – whatever separates the arms’ live URLLC/mMTC behavior (Section V-D) was not something the offline reward could

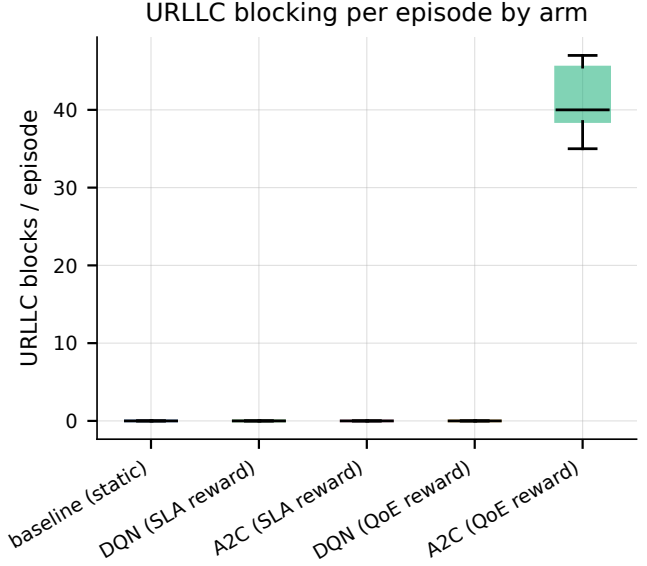


Fig. 6: URLLC agent-issued rejections/episode by arm, pooled across 3 seeds $\times$ 5 episodes. This is an internal admission-gate decision count, not a measured dropped PDU – see Section III-B.

have selected for. We report this plainly rather than only showing the (correct, but partial) eMBB success, whose offline config is calibrated against live-representative headroom: a frozen-weight policy that transfers perfectly on the one slice whose offline environment is representative of live conditions says nothing, by itself, about the two slices whose offline environment was never designed to be.

F. A preliminary, offline-only probe of within-episode demand variation

Every result above uses constant per-slice demand for the whole episode, both in live evaluation (Section IV) and in offline training (`sac1b_offline_campaign.yaml`). As a small, deliberately preliminary check – no live rig time, no retraining – we replayed each of the four already-frozen checkpoints through the same offline closed-loop simulator with a 3-phase within-episode demand schedule instead (steps 1–20 at $0.7\times$, 21–40 at $1.3\times$, 41–60 at $1.0\times$ each slice’s already-configured offline mean offered ratio; 3 training seeds $\times$ 5 episodes/arm, matching the live campaign’s own episode count). Fig. 8 shows the result: SLA compliance is low in every phase for every arm – including the $0.7\times$ (“low”, i.e. lighter than training-time demand) phase, where eMBB compliance is only 7–17% across the four arms, far below the 100.0% these same policies hold under constant demand (Table III) – and degrades further, monotonically, from low $\rightarrow$ high $\rightarrow$ medium. We do not have a confirmed causal account from this small probe alone: a plausible reading is that these slices’ ceilings have essentially zero elastic headroom in this offline environment (Section V-E), so backlog accumulated during the $1.3\times$ phase cannot be

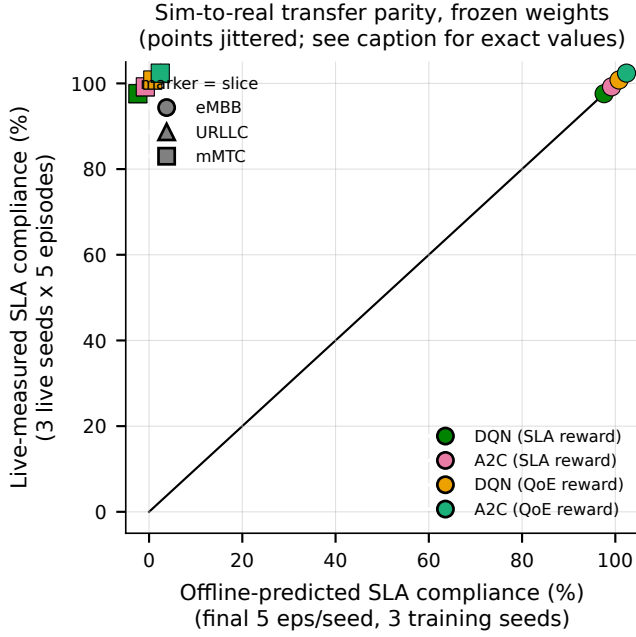


Fig. 7: Offline-predicted vs. live-measured SLA compliance, frozen weights, per arm $\times$ slice (points jittered per-arm for visibility; all 4 arms coincide exactly at both plotted locations per slice – see Section V-E for exact values). eMBB sits on the $y=x$ line; URLLC/mMTC sit at (offline= 0, live= 100) for every arm.

relieved by ceiling adjustment and persists into the following phase regardless of demand dropping back down; an equally plausible reading is that a policy trained to convergence against one fixed demand level has simply not seen, and does not generalize to, any other level. Distinguishing these (and retraining under an explicitly time-varying curriculum) is exactly the scope of the journal-length follow-up’s R1/R2 stages (experiments/REWORK_PLAN.md); this probe’s only claim is that constant-demand training does not transfer to within-episode demand variation on this rig’s environment, not why.

As a small, directional check that this is not purely a simulator artifact, we additionally ran the same 3-phase schedule *live* against real iperf3 traffic (rate switched at each phase boundary, synced to the live step clock) for 2 of the 4 arms (dqn_sla, a2c_qoe), 1 seed, 2 episodes each – a directional smoke check, not a powered comparison. eMBB compliance under live time-varying demand was 40.0%/25.0%/42.5% (low/high/medium) for dqn_sla and 2.5%/0.0%/0.0% for a2c_qoe, with URLLC/mMTC at or near 0% in every phase for both arms – all far below the 100.0% these same checkpoints hold under the constant-demand live campaign (Table III). The direction matches the offline probe; $n=2$ episodes/arm is far too small to say anything about the exact numbers. (Operational note: restarting all 3 slices’ iperf3 clients at each phase boundary transiently disrupted the UEs’

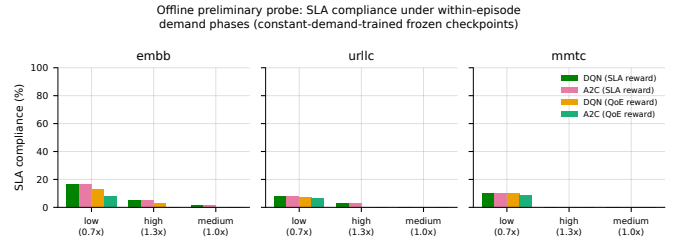


Fig. 8: Offline-only preliminary probe (no live rig time): SLA compliance per arm, per slice, under a 3-phase within-episode demand schedule applied to the four already-frozen, constant-demand-trained checkpoints. 3 training seeds $\times$ 5 episodes/arm.

data-plane reachability to external/target hosts after the run completed – the E2 control-plane path itself, which is what the reported compliance numbers depend on, was re-verified healthy throughout via the same precondition probe used before every live episode in this campaign, and no crash signature appeared in the kernel log; consistent with, not an addition to, the rig-fragility limitation already disclosed in Section VII.)

VI. CONGESTED MULTI-SLICE ADMISSION WITH URLLC PRIORITIZATION

Sections III–V evaluate all arms under one constant, per-slice demand level for the whole episode (Section V-F’s own preliminary probe already shows this doesn’t generalize to within-episode demand variation). This section reports a separate, deliberately preliminary, offline-only follow-up targeting a different, specific question: under genuinely congested, dynamic, and randomized multi-slice traffic, do the learned arms preserve URLLC’s SLA compliance ahead of eMBB/mMTC, consistent with URLLC already carrying this campaign’s highest reward priority (priority_weight=5.0, violation_penalty=8.0, both highest of the three slices, Table II)? This required diagnosing and fixing two structural gaps in the existing offline training setup, not just a new evaluation script.

A. Two structural gaps, fixed

First, the offline training config (sac1b_offline_campaign.yaml) pins every slice’s ceiling at its own nominal_ratio with zero elastic headroom (Section V-E), while the live-evaluation config has real headroom (12/4/3). A new training config, sac1b_offline_congested_v1.yaml, matches the live config’s cap values exactly, closing this train/eval mismatch. Second, and more consequentially: the offline synthetic KPM source (ClosedLoopKpmSource) computes each slice’s served PRBs independently – there is no constraint that the three slices’ served PRBs sum to a shared budget, so raising one slice’s ceiling never actually took capacity away from another in any offline result reported so far. A new SharedPoolCongestedKpmSource adds this directly: each slice’s ceiling-limited request is computed

TABLE IV: SLA compliance (%) under congested/dynamic/randomized traffic with genuine shared-PRB contention; held out seeds $(950/951/952) \times 15$ episodes/arm, frozen weights.

Arm	URLLC	eMBB	mMTC
baseline (static)	22.6	34.5	19.0
DQN (SLA reward)	32.2	42.4	14.7
A2C (SLA reward)	27.1	50.2	15.2
DQN (QoE reward)	39.0	7.6	7.7
A2C (QoE reward)	9.4	7.6	7.7

as before, and if the three slices’ combined request exceeds a shared pool (calibrated to 8 PRB – deliberately smaller than the three ceilings’ combined maximum of 19 PRB, so maxing out every ceiling simultaneously creates genuine scarcity), every slice’s actual served amount is scaled down proportionally to its own request share. This is a neutral, non-priority physical rationing rule – URLLC gets no special treatment in the physics itself, so any preferential protection observed downstream comes from the policy’s own reward-driven behavior, not from the simulator. Per-episode demand is additionally randomized (each slice independently samples an oversubscription multiplier in $[0.4 \times, 1.3 \times]$ its configured mean at the start of every episode) and remains dynamic within the episode via the KPM source’s existing mean-reverting random walk.

B. Contention-aware reward shaping

Training against the shared-pool mechanism directly (600 episodes, 3 seeds, otherwise identical protocol to Section III-E) surfaced a real failure mode before any fix: eMBB’s larger ceiling cap let it accumulate enough service-term reward from raw accept-volume to outweigh URLLC’s higher per-violation penalty, so the learned policy improved eMBB while leaving URLLC no better (and in two of four arms, worse) than baseline. Two changes address this directly: URLLC’s `violation_penalty` was raised from 8.0 to 20.0, and an additional shaping term subtracts $15 \times \text{contention_ratio}$ from the step reward whenever URLLC is in violation, where `contention_ratio` $\in [0, 1]$ is how saturated the shared pool is *at that step* – zero effect when the pool has slack, largest effect exactly when URLLC is failing under real scarcity. Both changes are documented in `sacrb_offline_congested_v1.yaml` and `train_offline_congested.py` respectively, with the failure mode that motivated them.

C. Result: DQN+QoE preserves URLLC by sacrificing the other two; A2C does not learn a differentiated policy

Fig. 9 and Table IV show the held-out result. DQN (QoE) is the only arm that both (a) improves URLLC above every other arm (22.6% $\rightarrow$ 39.0%, the largest URLLC gain of the four) and (b) does so by actively *sacrificing* the other two slices (34.5% $\rightarrow$ 7.6% eMBB, 19.0% $\rightarrow$ 7.7% mMTC) – a genuine priority tradeoff, not a policy that merely rides every ceiling upward. DQN (SLA) also improves URLLC

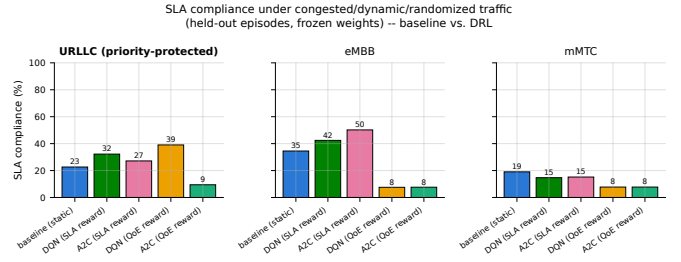


Fig. 9: SLA compliance under congested/dynamic/randomized traffic with genuine shared-PRB contention, per arm, URLLC panel bolded. DQN(QoE) is the only arm that both leads on URLLC and visibly sacrifices eMBB/mMTC to do it.

(22.6% $\rightarrow$ 32.2%) but without sacrificing eMBB ($\rightarrow$ 42.4%), a smaller and less differentiated effect. A2C (SLA) remains eMBB-biased (eMBB 50.2% vs. URLLC 27.1%), the same pattern observed before the contention-aware shaping fix.

A2C (QoE) is worse than baseline on all three slices. Direct inspection of frozen-weight action selection (not just aggregate compliance) explains why: both A2C arms converge to a *fully deterministic, state-independent* policy – `accept_frac` exactly 1.0 for A2C(SLA) and exactly 0.0 for A2C(QoE), on every slice, every state, confirmed by sampling actions directly rather than inferring this from compliance numbers alone. This did not change under either of two independent interventions: an external ϵ -greedy exploration override during training, or raising A2CPolicy’s entropy coefficient from its default 0.01 (a real, previously hardcoded constant, now configurable) up to 0.3 – the higher-entropy run converged to the *opposite* deterministic extreme (always-accept) faster (100 episodes) rather than to a differentiated, contention-aware policy. We read this as evidence that A2C, in this per-request binary-action framing, is finding the single action with the best *average* reward across all training states rather than learning to condition its decision on whether the current moment calls for protecting URLLC – an algorithm-dependent limitation, not a reward- or exploration-tuning problem, and we report it as such rather than continuing to search for a fix that changes this.

TODO(MANOJ): Scope statement for this section (your call): this is a new, offline-only, preliminary experiment with its own training config and KPM-source mechanism, methodologically separate from the live campaign in Sections III-V (not yet live-validated) – state this plainly and note what a live confirmation of the shared-PRB-contention mechanism would require (Section VII already discusses live rig fragility/scarcity). Also your call: whether A2C’s deterministic-collapse finding belongs here as a contribution (a real, reproducible algorithm-dependent limitation) or is better positioned as a limitation/future-work item.

VII. DISCUSSION

TODO(MANOJ): Discussion framing – suggested content sketch:

- What the live results confirm vs. what remains open relative to the simulation-only papers #1/#2.
- The QoE-vs-SLA-reward tradeoff tension, if the URLLC MOS finding replicates at multi-seed scale (Section V/VII will state the verdict; this section is where you interpret it for the reader).
- **New judgment call (Section V-E’s sim-to-real parity finding):** is the URLLC/mMTC offline-vs-live mismatch better framed as (a) a testbed-engineering limitation worth flagging honestly and fixing before the journal-length campaign (i.e., the offline environment’s demand/backlog scaling for these two slices needs recalibrating against live measurement, the way eMBB’s was), or (b) a positive robustness finding (the deployed policies did fine live despite an offline training signal that gave them no real compliance feedback for 2 of 3 slices on this reward)? Your call – the data supports either reading; Section V-E only reports what happened, not what it means for the paper’s claims.
- Positioning for the journal-length follow-up: GNN state representation, multi-agent coordination, and federated learning across gNBs are natural extensions once a multi-gNB physical rig exists – explicitly out of scope here. The Section V-E finding also motivates re-validating (or replacing) the offline training environment’s URLLC/mMTC demand scaling against live measurement before that follow-up’s larger campaign, the same way eMBB’s already matches.

VIII. LIMITATIONS

This work has the following limitations. **Radio interface.** All results are from OAI’s rfsim radio-frequency simulator, not an over-the-air deployment; rfsim reproduces the full protocol stack and real scheduler behavior but not physical-layer channel impairments. **Hardware.** The testbed runs on a single shared desktop machine (7.4 GB RAM, 8 cores), not an isolated telecom-grade rig; running the full gNB + 3-UE stack was found to be subject to an intermittent radio-link failure (RLC maximum-retransmission on the highest-traffic slice), plausibly caused by missed real-time scheduling deadlines under memory pressure rather than a logic fault. This was mitigated, not eliminated: a health-check-and-restart mechanism runs automatically between evaluation episodes and was verified live to recover automatically in every observed instance, at a wall-clock cost of roughly one restart per episode transition under full traffic load; every evaluation episode counted in Section V completed and was log-verified end to end, and no partial or corrupted episode was silently included. **Scale.** A single gNB and three UEs (one per slice) were used; the multi-gNB load-balancing term from paper #2 is consequently inapplicable here ($\rho \equiv 1$ with one gNB)

and is not evaluated. **Sample size.** Live evaluation used 3 seeds per arm; Section V reports variance honestly rather than treating $n = 3$ as sufficient for strong statistical claims. **QoE labels.** Per-slice MOS is an *inferred* quantity (IQX prior + LSTM refinement calibrated against P.1203/ACR proxies), not a measurement from real end-user subjective scoring.

REPRODUCIBILITY

ORANSlice commit b9bcc9b1 (OAI 2024.w28 base; full hash: b9bcc9b17becfc1041072a7b8d0f01ae874aba2). Evaluation seeds: 950, 951, 952 (live); offline-training seeds: 256, 257, 258. All configs (experiments/configs/*.yaml), orchestration scripts, and raw per-step logs (omega_log.jsonl, one row per control-loop step with a non-empty limitation field by construction) are preserved and version-controlled; every figure in this paper is regenerated directly from those logs by a committed script in experiments/plots/, never hand-plotted.

IX. CONCLUSION

TODO(MANOJ): Conclusion – 3-4 sentences summarizing the contribution and the single most important number/finding from Section V, plus a one-sentence pointer to the journal-length follow-up (paper #3’s full GNN-MARL-FL scope).

REFERENCES

- [1] WinesLab, “ORANSlice: O-RAN network slicing on OpenAirInterface,” <https://github.com/wineslab/ORANSlice>, oRANSlice fork, OAI 2024.w28 base, commit b9bcc9b. VERIFY: confirm citation format WinesLab/Northeastern prefers for this repo.
- [2] OpenAirInterface Software Alliance, “OpenAirInterface: 5g software alliance for democratising wireless innovation,” <https://openairinterface.org>, VERIFY: check for a canonical OAI citation (e.g. a EuCNC/WCNC demo paper) vs. the project URL.
- [3] O-RAN Alliance, “O-RAN working group 3, near-RT RIC architecture and E2 general aspects and principles (E2AP),” O-RAN Alliance, Tech. Rep., 2023, VERIFY: exact spec number/version (e.g. O-RAN.WG3.E2AP-v03.00) and year.
- [4] —, “O-RAN working group 3, E2 service model (E2SM) KPM,” O-RAN Alliance, Tech. Rep., 2023, VERIFY: exact spec number/version (e.g. O-RAN.WG3.E2SM-KPM-v03.00) and year.
- [5] M. Fiedler, T. Hossfeld, and P. Tran-Gia, “The IQX hypothesis: A fundamental property of quality of experience,” *IEEE Network*, 2010, VERIFY: exact volume/issue/page numbers.
- [6] ITU-T, “P.1203: Parametric bitstream-based quality assessment of progressive download and adaptive audiovisual streaming services over reliable transport,” ITU-T, Tech. Rep., 2017, VERIFY: exact edition/amendment referenced (base P.1203 vs. P.1203.1/2/3).